C++ – Module 02

Polymorphisme ad-hoc, surcharge d'opérateurs
et Orthodox Canonical class form

Résumé : ce document contient les exercices du Module 02 des modules C++.
Version : 9.1 – Traduction française

Sommaire

I Introduction

II Règles générales

III Nouvelles règles

IV Instructions sur l'IA

V Exercice 00 : My First Class in Orthodox Canonical Form

VI Exercice 01 : Towards a more useful fixed-point number class

VII Exercice 02 : Now we're talking

VIII Exercice 03 : BSP

IX Rendu et Peer-Evaluation

Chapitre I – Introduction

C++ est un langage de programmation généraliste créé par Bjarne Stroustrup comme une extension du langage C, ou « C with Classes » (source : Wikipedia).

Le but de ces modules est de t'initier à la **Programmation Orientée Objet** (OOP). Ce sera le point de départ de ton aventure C++. Beaucoup de langages sont recommandés pour apprendre l'OOP, mais nous avons choisi C++ parce qu'il vient de ton vieil ami le C. Comme c'est un langage complexe, et pour garder les choses simples, ton code devra respecter le standard C++98.

Nous savons que le C++ moderne est assez différent sur de nombreux points. Donc, si tu veux devenir un développeur C++ compétent, c'est à toi d'aller plus loin après le Common Core de 42 !

Chapitre II – Règles générales

Compilation

- Compile ton code avec `c++` et les flags `-Wall -Wextra -Werror`
- Ton code doit toujours compiler si tu ajoutes le flag `-std=c++98`

Formatage et conventions de nommage

- Les dossiers des exercices s'appellent : `ex00`, `ex01`, ... , `exn`
- Nomme tes fichiers, classes, fonctions, member functions et attributs comme demandé dans les consignes.
- Écris les noms de classes en **UpperCamelCase**. Les fichiers contenant le code d'une classe portent toujours le nom de la classe. Par exemple : `ClassName.hpp`/`ClassName.h`, `ClassName.cpp` ou `ClassName.tpp`. Si tu as un header qui contient la définition d'une classe « BrickWall » (un mur de briques), il s'appellera `BrickWall.hpp`.
- Sauf indication contraire, chaque message affiché doit se terminer par un retour à la ligne (newline) et être affiché sur la sortie standard.
- *Goodbye Norminette !* Aucun style de code n'est imposé dans les modules C++. Tu peux suivre ton style préféré. Mais garde en tête que du code que tes évaluateurs ne comprennent pas est du code qu'ils ne peuvent pas noter. Fais de ton mieux pour écrire un code propre et lisible.

Autorisé / Interdit

Tu ne codes plus en C. C'est l'heure du C++ ! Donc :

- Tu peux utiliser presque tout ce qu'il y a dans la standard library. Donc, au lieu de rester sur ce que tu connais déjà, il serait malin d'utiliser autant que possible les versions « C++-ish » des fonctions C que tu as l'habitude d'utiliser.
- Par contre, tu ne peux utiliser aucune autre bibliothèque externe. Ça veut dire que C++11 (et les formes dérivées) et les bibliothèques Boost sont interdits. Ces fonctions sont aussi interdites : `*printf()`, `*alloc()` et `free()`. Si tu les utilises, ta note sera 0, et c'est tout.
- Sauf indication explicite contraire, les mots-clés `using namespace <ns_name>` et `friend` sont interdits. Sinon, ta note sera -42.
- **Tu as le droit d'utiliser la STL seulement dans les Modules 08 et 09.** Ça veut dire : pas de **Containers** (vector/list/map, etc.) et pas d'**Algorithms** (tout ce qui demande

d'inclure le header `<algorithm>`) jusque-là. Sinon, ta note sera -42.

Quelques contraintes de conception

- Les fuites de mémoire existent aussi en C++. Quand tu alloues de la mémoire (avec le mot-clé `new`), tu dois éviter les **memory leaks**.
- Du Module 02 au Module 09, tes classes doivent être conçues en **Orthodox Canonical Form, sauf indication explicite contraire**.
- Toute implémentation de fonction mise dans un header (sauf pour les function templates) = 0 à l'exercice.
- Tu dois pouvoir utiliser chacun de tes headers indépendamment des autres. Ils doivent donc inclure toutes leurs dépendances. Par contre, tu dois éviter le problème de la double inclusion avec des **include guards**. Sinon, ta note sera 0.

Read me

- Tu peux ajouter des fichiers supplémentaires si besoin (par exemple pour découper ton code). Comme ces exercices ne sont pas vérifiés par un programme, tu es libre de le faire tant que tu rends les fichiers obligatoires.
- Parfois, les consignes d'un exercice semblent courtes, mais les exemples peuvent montrer des exigences qui ne sont pas écrites explicitement.
- Lis chaque module en entier avant de commencer ! Vraiment, fais-le.
- Par Odin, par Thor ! Utilise ton cerveau !!!

Attention – Pour le Makefile des projets C++, les mêmes règles qu'en C s'appliquent (voir le chapitre de la Norme sur le Makefile).

Astuce – Tu vas devoir implémenter beaucoup de classes. Ça peut sembler pénible, sauf si tu sais scripter ton éditeur de texte préféré.

Info – Tu as une certaine liberté pour réaliser les exercices. Mais respecte les règles obligatoires et ne sois pas paresseuse. Tu passerais à côté de beaucoup d'informations utiles ! N'hésite pas à lire sur les concepts théoriques.

Chapitre III – Nouvelles règles

Désormais, toutes tes classes doivent être conçues en **Orthodox Canonical Form**, sauf indication explicite contraire. Elles devront donc implémenter les quatre member functions obligatoires ci-dessous :

- Default constructor
- Copy constructor
- Copy assignment operator
- Destructor

Sépare le code de ta classe en deux fichiers. Le fichier header (.hpp/.h) contient la définition de la classe, et le fichier source (.cpp) contient l'implémentation.

Chapitre IV – Instructions sur l'IA

● Contexte

Ce projet est conçu pour t'aider à découvrir les bases fondamentales de ta formation 42.

Pour bien ancrer les connaissances et compétences clés, il est essentiel d'adopter une approche réfléchie de l'utilisation des outils d'IA et de leur soutien.

Un apprentissage fondamental solide demande un vrai effort intellectuel : par le défi, la répétition et les échanges entre pairs (peer-learning).

Pour un aperçu plus complet de notre position sur l'IA (comme outil d'apprentissage, dans la formation 42, et comme attente du marché du travail), consulte la FAQ dédiée sur l'intranet.

● Message principal

- ☛ Construis des bases solides, sans raccourcis.
- ☛ Développe vraiment tes compétences techniques et tes « power skills ».
- ☛ Vis un vrai peer-learning, apprends à apprendre et à résoudre de nouveaux problèmes.
- ☛ Le parcours d'apprentissage est plus important que le résultat.
- ☛ Découvre les risques liés à l'IA, et développe de bonnes pratiques de contrôle et des contre-mesures pour éviter les pièges courants.

● Règles de l'apprenant

- Tu dois réfléchir à tes tâches, surtout avant de te tourner vers l'IA.
- Tu ne dois pas demander de réponses directes à l'IA.
- Tu dois découvrir l'approche globale de 42 sur l'IA.

● Résultats de la phase

Dans cette phase fondamentale, tu vas obtenir les résultats suivants :

- Acquérir de bonnes bases techniques et de code.
- Savoir pourquoi et comment l'IA peut être dangereuse pendant cette phase.

● Commentaires et exemples

- Oui, on sait que l'IA existe, et oui, elle peut résoudre tes projets. Mais tu es ici pour apprendre, pas pour prouver que l'IA a appris. Ne perds pas ton temps (ni le nôtre) juste

pour montrer que l'IA sait résoudre le problème donné.

- Apprendre à 42, ce n'est pas connaître la réponse : c'est développer la capacité d'en trouver une. L'IA te donne directement la réponse, mais ça t'empêche de construire ton propre raisonnement. Et le raisonnement demande du temps, de l'effort, et passe par l'échec. Le chemin vers le succès n'est pas censé être facile.
- Garde en tête que pendant les exams, l'IA n'est pas disponible : pas d'internet, pas de smartphones, etc. Tu verras vite si tu t'es trop appuyée sur l'IA dans ton apprentissage.
- Le peer-learning t'expose à des idées et des approches différentes, améliore tes compétences relationnelles et ta capacité à penser autrement. C'est bien plus précieux que de simplement discuter avec un bot. Alors n'aie pas peur : parle, pose des questions, et apprenez ensemble !
- Oui, l'IA fera partie du cursus, à la fois comme outil d'apprentissage et comme sujet en soi. Tu auras même la chance de construire ton propre logiciel d'IA. Pour en savoir plus sur notre approche « crescendo », consulte la documentation disponible sur l'intranet.

✓ **Bonne pratique** – Je suis bloquée sur un nouveau concept. Je demande à quelqu'un à côté comment il s'y est pris. On discute 10 minutes, et tout à coup ça fait « clic ». Je comprends.

✗ **Mauvaise pratique** – J'utilise l'IA en cachette, je copie du code qui a l'air juste. Pendant la peer evaluation, je ne sais rien expliquer. Je rate. Pendant l'exam, sans IA, je suis encore bloquée. Je rate.

Chapitre V – Exercice 00 : My First Class in Orthodox Canonical Form

Dossier	`ex00/`
Fichiers à rendre	`Makefile`, `main.cpp`, `Fixed.{h, hpp}`, `Fixed.cpp`
Interdit	Rien

Tu penses connaître les integers et les floating-point numbers. Comme c'est mignon.

Lis cet article de 3 pages (1, 2, 3) pour découvrir que ce n'est pas le cas. Vas-y, lis-le.

Jusqu'à aujourd'hui, tous les nombres que tu as utilisés dans ton code étaient en gros soit des integers, soit des floating-point numbers, ou une de leurs variantes (`short`, `char`, `long`, `double`, etc.). Après avoir lu l'article, on peut supposer que les integers et les floating-point numbers ont des caractéristiques opposées.

Mais aujourd'hui, ça change. Tu vas découvrir un nouveau type de nombre génial : les **fixed-point numbers** ! Absents de presque tous les types scalaires des langages, les fixed-point numbers offrent un bon équilibre entre performance, exactitude, étendue (range) et précision. C'est pour ça qu'ils sont très utilisés en infographie (computer graphics), en traitement du son ou en programmation scientifique, pour n'en citer que quelques-uns.

Comme C++ n'a pas de fixed-point numbers, tu vas les ajouter. Cet article de Berkeley est un bon début. Si tu ne sais pas ce qu'est l'université de Berkeley, lis cette section de sa page Wikipedia.

Crée une classe en Orthodox Canonical Form qui représente un fixed-point number :

Membres privés

- Un **integer** pour stocker la valeur du fixed-point number.
- Un **static constant integer** pour stocker le nombre de fractional bits. Sa valeur sera toujours le littéral entier `8`.

Membres publics

- Un default constructor qui initialise la valeur du fixed-point number à `0`.
- Un copy constructor.
- Un copy assignment operator overload.
- Un destructor.
- Une member function `int getRawBits( void ) const;` qui retourne la valeur brute (raw) du fixed-point number.
- Une member function `void setRawBits( int const raw );` qui définit la valeur brute (raw) du fixed-point number.

Lancer ce code :

```
#include <iostream>

int main( void ) {

    Fixed a;
    Fixed b( a );
    Fixed c;

    c = b;

    std::cout << a.getRawBits() << std::endl;
    std::cout << b.getRawBits() << std::endl;
    std::cout << c.getRawBits() << std::endl;

    return 0;
}
```

Doit afficher quelque chose de similaire à :

```
$> ./a.out
Default constructor called
Copy constructor called
Copy assignment operator called // <-- Cette ligne peut manquer selon ton implémentation
getRawBits member function called
Default constructor called
Copy assignment operator called
getRawBits member function called
getRawBits member function called
0
getRawBits member function called
0
getRawBits member function called
0
Destructor called
Destructor called
Destructor called
$>
```

Chapitre VI – Exercice 01 : Towards a more useful fixed-point number class

Dossier	`ex01/`
Fichiers à rendre	`Makefile`, `main.cpp`, `Fixed.{h, hpp}`, `Fixed.cpp`
Autorisé	`roundf` (de `<cmath>`)

L'exercice précédent était un bon début, mais notre classe est assez inutile. Elle peut seulement représenter la valeur `0.0`.

Ajoute les constructors et member functions publics suivants à ta classe :

- Un constructor qui prend un **constant integer** en paramètre. Il le convertit en la valeur fixed-point correspondante. La valeur des fractional bits doit être initialisée à 8, comme dans l'exercice 00.
- Un constructor qui prend un **constant floating-point number** en paramètre. Il le convertit en la valeur fixed-point correspondante. La valeur des fractional bits doit être initialisée à 8, comme dans l'exercice 00.
- Une member function `float toFloat( void ) const;` qui convertit la valeur fixed-point en valeur floating-point.
- Une member function `int toInt( void ) const;` qui convertit la valeur fixed-point en valeur integer.

Et ajoute la fonction suivante aux fichiers de la classe **Fixed** :

- Une surcharge de l'opérateur d'insertion (`<<`) qui insère une représentation floating-point du fixed-point number dans l'output stream passé en paramètre.

Lancer ce code :

```
#include <iostream>

int main( void ) {

    Fixed       a;
    Fixed const b( 10 );
    Fixed const c( 42.42f );
    Fixed const d( b );

    a = Fixed( 1234.4321f );

    std::cout << "a is " << a << std::endl;
    std::cout << "b is " << b << std::endl;
    std::cout << "c is " << c << std::endl;
    std::cout << "d is " << d << std::endl;

    std::cout << "a is " << a.toInt() << " as integer" << std::endl;
    std::cout << "b is " << b.toInt() << " as integer" << std::endl;
    std::cout << "c is " << c.toInt() << " as integer" << std::endl;
    std::cout << "d is " << d.toInt() << " as integer" << std::endl;

    return 0;
}
```

Doit afficher quelque chose de similaire à :

```
$> ./a.out
Default constructor called
Int constructor called
Float constructor called
Copy constructor called
Copy assignment operator called
Float constructor called
Copy assignment operator called
Destructor called
a is 1234.43
b is 10
c is 42.4219
d is 10
a is 1234 as integer
b is 10 as integer
c is 42 as integer
d is 10 as integer
Destructor called
Destructor called
Destructor called
Destructor called
$>
```

Chapitre VII – Exercice 02 : Now we're talking

Dossier	`ex02/`
Fichiers à rendre	`Makefile`, `main.cpp`, `Fixed.{h, hpp}`, `Fixed.cpp`
Autorisé	`roundf` (de `<cmath>`)

Ajoute des member functions publiques à ta classe pour surcharger les opérateurs suivants :

- Les 6 opérateurs de comparaison : `>`, `<`, `>=`, `<=`, `==` et `!=`.
- Les 4 opérateurs arithmétiques : `+`, `-`, `*` et `/`.
- Les 4 opérateurs d'incrémentation/décrémentation (pré-incrémentation et post-incrémentation, pré-décrémentation et post-décrémentation), qui augmentent ou diminuent la valeur fixed-point du plus petit epsilon (ϵ) représentable, tel que $1 + \epsilon > 1$.

Ajoute ces quatre member functions publiques surchargées à ta classe :

- Une static member function `min` qui prend deux références vers des fixed-point numbers en paramètres, et retourne une référence vers le plus petit.
- Une static member function `min` qui prend deux références vers des fixed-point numbers **constants** en paramètres, et retourne une référence vers le plus petit.
- Une static member function `max` qui prend deux références vers des fixed-point numbers en paramètres, et retourne une référence vers le plus grand.
- Une static member function `max` qui prend deux références vers des fixed-point numbers **constants** en paramètres, et retourne une référence vers le plus grand.

C'est à toi de tester toutes les fonctionnalités de ta classe. Cependant, lancer le code ci-dessous :

```
#include <iostream>

int main( void ) {

    Fixed       a;
    Fixed const b( Fixed( 5.05f ) * Fixed( 2 ) );

    std::cout << a << std::endl;
    std::cout << ++a << std::endl;
    std::cout << a << std::endl;
    std::cout << a++ << std::endl;
    std::cout << a << std::endl;

    std::cout << b << std::endl;

    std::cout << Fixed::max( a, b ) << std::endl;

    return 0;
}
```

Doit afficher quelque chose comme ça (pour plus de lisibilité, les messages de constructor/destructor sont enlevés dans l'exemple ci-dessous) :

```
$> ./a.out
0
0.00390625
0.00390625
0.00390625
0.0078125
10.1016
10.1016
$>
```

Info – Si tu fais une division par 0, il est acceptable que le programme crashe.

Chapitre VIII – Exercice 03 : BSP

Dossier	`ex03/`
Fichiers à rendre	`Makefile`, `main.cpp`, `Fixed.{h, hpp}`, `Fixed.cpp`, `Point.{h, hpp}`, `Point.cpp`, `bsp.cpp`
Autorisé	`roundf` (de `<cmath>`)

Maintenant que tu as une classe **Fixed** fonctionnelle, ce serait bien de l'utiliser.

Implémente une fonction qui indique si un point est à l'intérieur d'un triangle ou non. Très utile, non ?

Astuce – BSP veut dire Binary Space Partitioning. De rien. :)

Info – Tu peux valider ce module sans faire l'exercice 03.

Commençons par créer la classe **Point** en Orthodox Canonical Form, qui représente un point 2D :

Membres privés

- Un attribut `Fixed const` nommé `x`.
- Un attribut `Fixed const` nommé `y`.
- Tout ce qui est utile en plus.

Membres publics

- Un default constructor qui initialise `x` et `y` à 0.
- Un constructor qui prend deux constant floating-point numbers en paramètres. Il initialise `x` et `y` avec ces paramètres.
- Un copy constructor.
- Un copy assignment operator overload.
- Un destructor.
- Tout ce qui est utile en plus.

Pour finir, implémente la fonction suivante dans le fichier approprié :

```
bool bsp( Point const a, Point const b, Point const c, Point const point);
```

- `a`, `b`, `c` : les sommets (vertices) de notre cher triangle.
- `point` : le point à vérifier.
- **Retourne** : `True` si le point est à l'intérieur du triangle. `False` sinon. Donc, si le point est un sommet ou est sur un côté (edge), la fonction retourne `False`.

Implémente et rends tes propres tests pour t'assurer que ta classe se comporte comme prévu.

Chapitre IX – Rendu et Peer-Evaluation

Rends ton travail dans ton dépôt **Git**, comme d'habitude. Seul le travail présent dans ton dépôt sera évalué pendant la défense. N'hésite pas à revérifier les noms de tes dossiers et fichiers pour être sûre qu'ils sont corrects.

Pendant l'évaluation, une courte **modification du projet** peut parfois être demandée. Ça peut être un petit changement de comportement, quelques lignes de code à écrire ou réécrire, ou une fonctionnalité facile à ajouter.

Cette étape n'est **pas forcément applicable à tous les projets**, mais tu dois y être préparée si elle est mentionnée dans les consignes d'évaluation.

Cette étape sert à vérifier que tu as vraiment compris une partie précise du projet. La modification peut se faire dans n'importe quel environnement de développement de ton choix (par exemple ton installation habituelle), et elle doit être faisable en quelques minutes, sauf si une durée précise est définie dans l'évaluation. Tu peux par exemple devoir faire une petite mise à jour d'une fonction ou d'un script, modifier un affichage, ou adapter une structure de données pour stocker une nouvelle information, etc.

Les détails (portée, cible, etc.) seront précisés dans les **consignes d'évaluation** et peuvent varier d'une évaluation à l'autre pour le même projet.